Listing 11.4 Drawing a map with our sample data on it

```
d3.json("world.geojson", function(data) {createMap(data)});

function createMap(countries) {
  var projection = d3.geo.mercator()
       .scale(100).translate([250,250])

  var geoPath = d3.geo.path().projection(projection);
  var g = d3.select("svg").append("g");

  g.selectAll("path.country")
       .data(countries.features)
       .enter()
       .append("path")
       .attr("d", geoPath)
       .attr("class", "country");

  g.selectAll("path.sample")
       .data(sampleData)
       .enter()
       .append("path")
       .attr("d", geoPath)
       .attr("class", "sample");

};
```

Adjusts the projection and translation of the projection rather than the <g> so we can use the projection later to draw to canvas

Although our random triangles will obviously be in different places, our code should still produce something that looks like figure 11.2.

A thousand datapoints isn't very many, even on a small map like this. And in any browser that supports SVG, the data should be able to render quickly and provide you with the kind of functionality, like mouseover and click events, that you may want from your data display. But if you add zoom controls, like you see in listing 11.5 (the same zooming we had in chapter 7), then you'll notice that the performance of

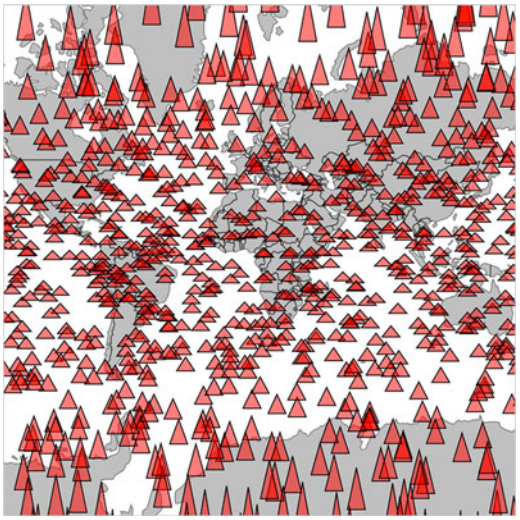


Figure 11.2 Drawing random triangles on a map entirely with SVG